Name : Patel Megh B **Er No**.: 22162121011 **Sem** : 7 **Subject** : CGC

Mini Project : Object Classifier (Binary Class Classification)

Problem Statement :

Object recognition plays an important role in cognitive computing and human-computer interaction. Humans can look at an object once and easily identify it again later, but machines must be trained to achieve the same capability.

The aim of this project is to design a system that:

1. captures live images through a webcam,
2. allows users to store images of two object categories as training data,
3. trains a machine-learning model to differentiate between those two objects, and
4. predicts and displays the object class during real-time camera input.

Key challenges addressed include:

- extracting reliable features from camera images,
- building a machine-learning model that generalizes from limited data,
- developing an intuitive interface for training and prediction,
- performing predictions fast enough for real-time usability.

Results and Interpretation:

Observations from live testing:

Number of Samples per Class	Approx. Accuracy
5 samples	Low accuracy, unstable predictions
10–20 samples	Good accuracy, stable predictions
30+ samples	Very good accuracy, consistent predictions

Key insights:

- The more diverse the training samples (different angles, lighting, distance), the better the performance.
- The **Linear SVM model** works well for binary classification with limited training data.

Future Improvements:

Although the current version performs well for two-class object recognition, the system can be significantly enhanced by incorporating the following improvements:

Enhancement	Description
Multiclass Classification	Extend support beyond 2 categories to 5–10 objects
Model Confidence Scores	Display probability/confidence instead of only labels
Transfer Learning	Replace SVM with CNN models (e.g., MobileNet, VGG16) for higher accuracy
Continuous Learning	Automatically add images when prediction is incorrect and retrain
Dataset Storage	Save datasets and trained model for reuse without retraining every session
Object Bounding Box	Add object detection to detect and classify multiple objects in the frame

Code :

Camera.py :

```
import cv2 as cv


class Camera:
    def __init__(self):
        self.camera = cv.VideoCapture(0)
        if not self.camera.isOpened():
            raise ValueError("Unable to open camera!")

        self.width = self.camera.get(cv.CAP_PROP_FRAME_WIDTH)
        self.height = self.camera.get(cv.CAP_PROP_FRAME_HEIGHT)

    def __del__(self):
        if self.camera.isOpened():
            self.camera.release()

    def get_frame(self):
        """
        Returns (ret, frame)
        - ret: True/False
```

```
        - frame: RGB image (numpy array) if ret is True, else None
        """
        if self.camera.isOpened():
            ret, frame = self.camera.read()

            if ret:
                # convert BGR (OpenCV default) to RGB (for Tkinter / PIL)
                return ret, cv.cvtColor(frame, cv.COLOR_BGR2RGB)
            else:
                return ret, None
        else:
            return False, None
```

App.py:

```
import tkinter as tk
from tkinter import simpledialog
import cv2 as cv
import os
from PIL import Image, ImageTk

import model
import camera


class App:
    def __init__(self, window_title="Camera Classifier"):
        # Create main window
        self.window = tk.Tk()
        self.window_title = window_title
        self.window.title(self.window_title)

        # Image counters for class 1 and 2
        self.counters = [1, 1]

        # Model and camera
        self.model = model.Model()
        self.auto_predict = False
        self.camera = camera.Camera()

        # Build GUI
        self.init_gui()

        # Update loop delay (ms)
        self.delay = 15
        self.update()

        # Keep window on top
```

```
        self.window.attributes("-topmost", True)
        self.window.mainloop()

    def init_gui(self):
        # Canvas to show camera feed
        self.canvas = tk.Canvas(
            self.window,
            width=int(self.camera.width),
            height=int(self.camera.height)
        )
        self.canvas.pack()

        # Auto prediction toggle
        self.btn_toggleauto = tk.Button(
            self.window,
            text="Auto Prediction",
            width=50,
            command=self.auto_predict_toggle
        )
        self.btn_toggleauto.pack(anchor=tk.CENTER, expand=True)

        # Ask for class names
        self.classname_one = simpledialog.askstring(
            "Classname One", "Enter the name of the first class:",
parent=self.window
        )
        self.classname_two = simpledialog.askstring(
            "Classname Two", "Enter the name of the second class:",
parent=self.window
        )

        # Fallbacks if user closes dialog / presses cancel
        if not self.classname_one:
            self.classname_one = "Class 1"
        if not self.classname_two:
            self.classname_two = "Class 2"

        # Buttons to capture images for each class
        self.btn_class_one = tk.Button(
            self.window,
            text=self.classname_one,
            width=50,
            command=lambda: self.save_for_class(1)
        )
        self.btn_class_one.pack(anchor=tk.CENTER, expand=True)

        self.btn_class_two = tk.Button(
            self.window,
```

```
            text=self.classname_two,
            width=50,
            command=lambda: self.save_for_class(2)
        )
        self.btn_class_two.pack(anchor=tk.CENTER, expand=True)

        # Train model button
        self.btn_train = tk.Button(
            self.window,
            text="Train Model",
            width=50,
            command=lambda: self.model.train_model(self.counters)
        )
        self.btn_train.pack(anchor=tk.CENTER, expand=True)

        # Single prediction button
        self.btn_predict = tk.Button(
            self.window,
            text="Predict",
            width=50,
            command=self.predict
        )
        self.btn_predict.pack(anchor=tk.CENTER, expand=True)

        # Reset button
        self.btn_reset = tk.Button(
            self.window,
            text="Reset",
            width=50,
            command=self.reset
        )
        self.btn_reset.pack(anchor=tk.CENTER, expand=True)

        # Label to show predicted class
        self.class_label = tk.Label(self.window, text="CLASS")
        self.class_label.config(font=("Arial", 20))
        self.class_label.pack(anchor=tk.CENTER, expand=True)

    def auto_predict_toggle(self):
        self.auto_predict = not self.auto_predict
        print("Auto prediction:", self.auto_predict)

    def save_for_class(self, class_num: int):
        """
        Capture current frame, convert to grayscale, resize to 150x150,
        and save to folder '1' or '2' depending on class_num.
        """
        ret, frame = self.camera.get_frame()
```

```python
        if not ret or frame is None:
            print("Failed to capture frame for saving.")
            return

        # Ensure class folders exist
        if not os.path.exists("1"):
            os.mkdir("1")
        if not os.path.exists("2"):
            os.mkdir("2")

        filename = f'{class_num}/frame{self.counters[class_num - 1]}.jpg'

        # Convert to grayscale and resize to fixed size (same as model)
        gray = cv.cvtColor(frame, cv.COLOR_RGB2GRAY)
        gray_resized = cv.resize(
            gray,
            (self.model.img_width, self.model.img_height)
        )

        cv.imwrite(filename, gray_resized)

        print(f"Saved: {filename}")
        self.counters[class_num - 1] += 1

    def reset(self):
        """
        Delete all saved images for both classes and reset counters + model.
        """
        for folder in ['1', '2']:
            if os.path.isdir(folder):
                for file in os.listdir(folder):
                    file_path = os.path.join(folder, file)
                    if os.path.isfile(file_path):
                        os.unlink(file_path)

        self.counters = [1, 1]
        self.model = model.Model()
        self.class_label.config(text="CLASS")
        print("Reset completed: images deleted and model reinitialized.")

    def update(self):
        """
        Periodically called to update camera frame on the canvas
        and (optionally) run auto prediction.
        """
        if self.auto_predict:
            prediction = self.predict()
            # prediction already prints via predict(); here we just trigger it
```

```
        ret, frame = self.camera.get_frame()

        if ret and frame is not None:
            img = Image.fromarray(frame)
            self.photo = ImageTk.PhotoImage(image=img)
            self.canvas.create_image(0, 0, image=self.photo, anchor=tk.NW)

        self.window.after(self.delay, self.update)

    def predict(self):
        """
        Capture a frame from the camera and use the model to predict the
class.
        """
        ret, frame = self.camera.get_frame()
        if not ret or frame is None:
            print("Failed to capture frame for prediction.")
            return

        prediction = self.model.predict(frame)

        if prediction == 1:
            self.class_label.config(text=self.classname_one)
            print("Predicted:", self.classname_one)
            return self.classname_one
        elif prediction == 2:
            self.class_label.config(text=self.classname_two)
            print("Predicted:", self.classname_two)
            return self.classname_two
        else:
            self.class_label.config(text="Unknown")
            print("Predicted: Unknown")
            return "Unknown"
```

Model.py :

```
from sklearn.svm import LinearSVC
import numpy as np
import cv2 as cv


class Model:
    def __init__(self):
        self.model = LinearSVC()
        self.is_trained = False

        # Fixed image size for all training & prediction
```

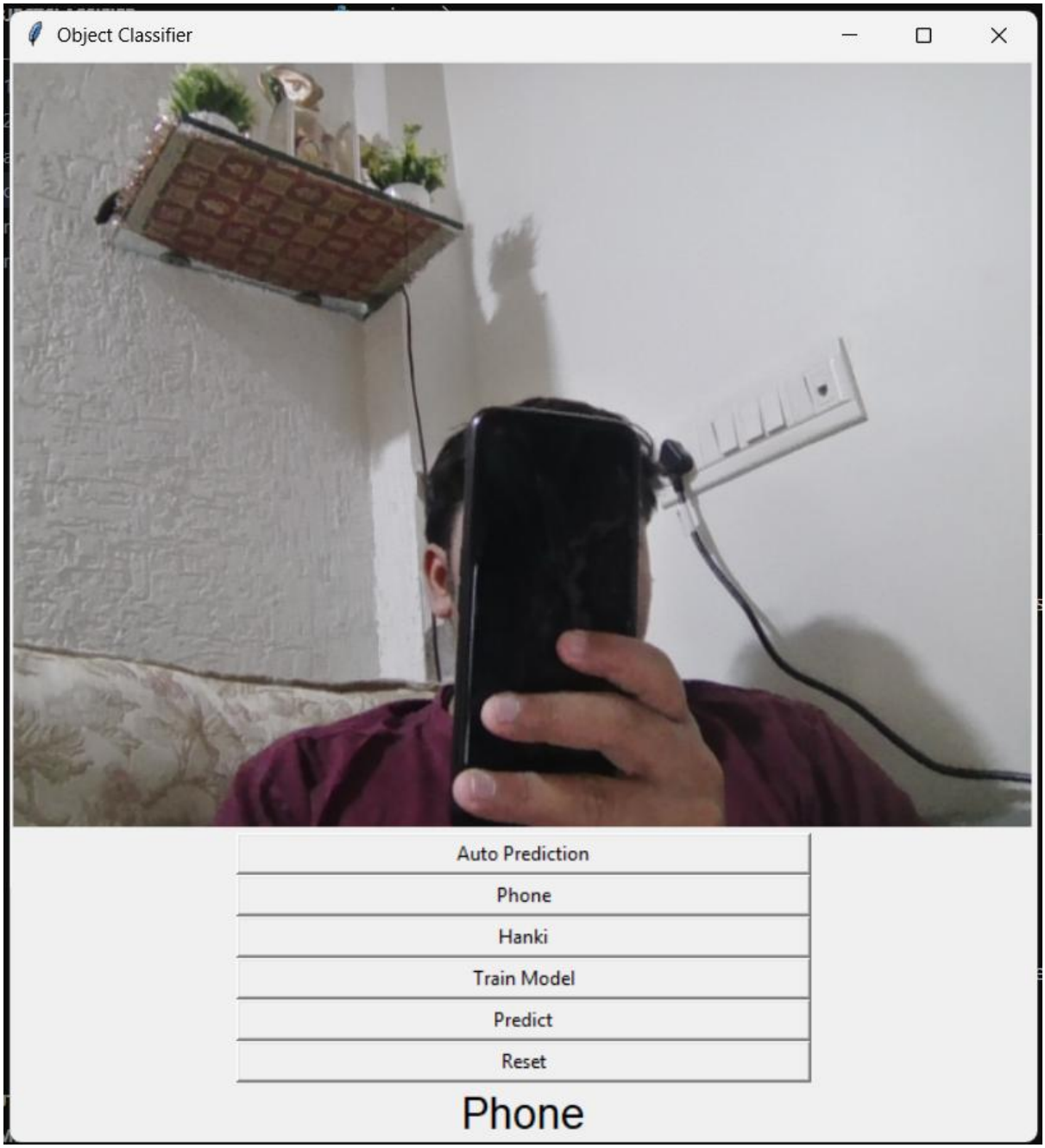
```python
        self.img_width = 150
        self.img_height = 150
        self.flat_size = self.img_width * self.img_height

    def _load_and_flatten(self, path):
        """
        Load image, convert to grayscale, resize to fixed size,
        and return flattened 1D numpy array.
        """
        img = cv.imread(path, cv.IMREAD_GRAYSCALE)
        if img is None:
            raise ValueError(f"Could not read image: {path}")

        img = cv.resize(img, (self.img_width, self.img_height))
        return img.reshape(self.flat_size)

    def train_model(self, counters):
        """
        Train SVM using images in folders '1' and '2'.
        counters: [count_for_class1, count_for_class2]
        """
        img_list = []
        class_list = []

        # Class 1 images
        for i in range(1, counters[0]):
            path = f'1/frame{i}.jpg'
            try:
                img_flat = self._load_and_flatten(path)
            except ValueError as e:
                print(e)
                continue
            img_list.append(img_flat)
            class_list.append(1)

        # Class 2 images
        for i in range(1, counters[1]):
            path = f'2/frame{i}.jpg'
            try:
                img_flat = self._load_and_flatten(path)
            except ValueError as e:
                print(e)
                continue
            img_list.append(img_flat)
            class_list.append(2)

        if not img_list:
            print("No training images found! Capture some images first.")
```

```
            return

        img_array = np.array(img_list)
        class_array = np.array(class_list)

        self.model.fit(img_array, class_array)
        self.is_trained = True
        print("Model successfully trained!")
        print("Samples:", img_array.shape[0], "Feature length:",
img_array.shape[1])

    def predict(self, frame):
        """
        Predict class for a given frame from the camera.
        frame: numpy array, RGB (from camera) or grayscale.
        """
        if not self.is_trained:
            print("Warning: Model not trained yet!")
            return 0  # "Unknown"

        # If color image, convert to grayscale
        if len(frame.shape) == 3:
            # frame is assumed RGB from camera.get_frame()
            gray = cv.cvtColor(frame, cv.COLOR_RGB2GRAY)
        else:
            gray = frame

        # Resize to fixed size and flatten
        gray_resized = cv.resize(gray, (self.img_width, self.img_height))
        img_flat = gray_resized.reshape(1, self.flat_size)

        prediction = self.model.predict(img_flat)
        return prediction[0]
```

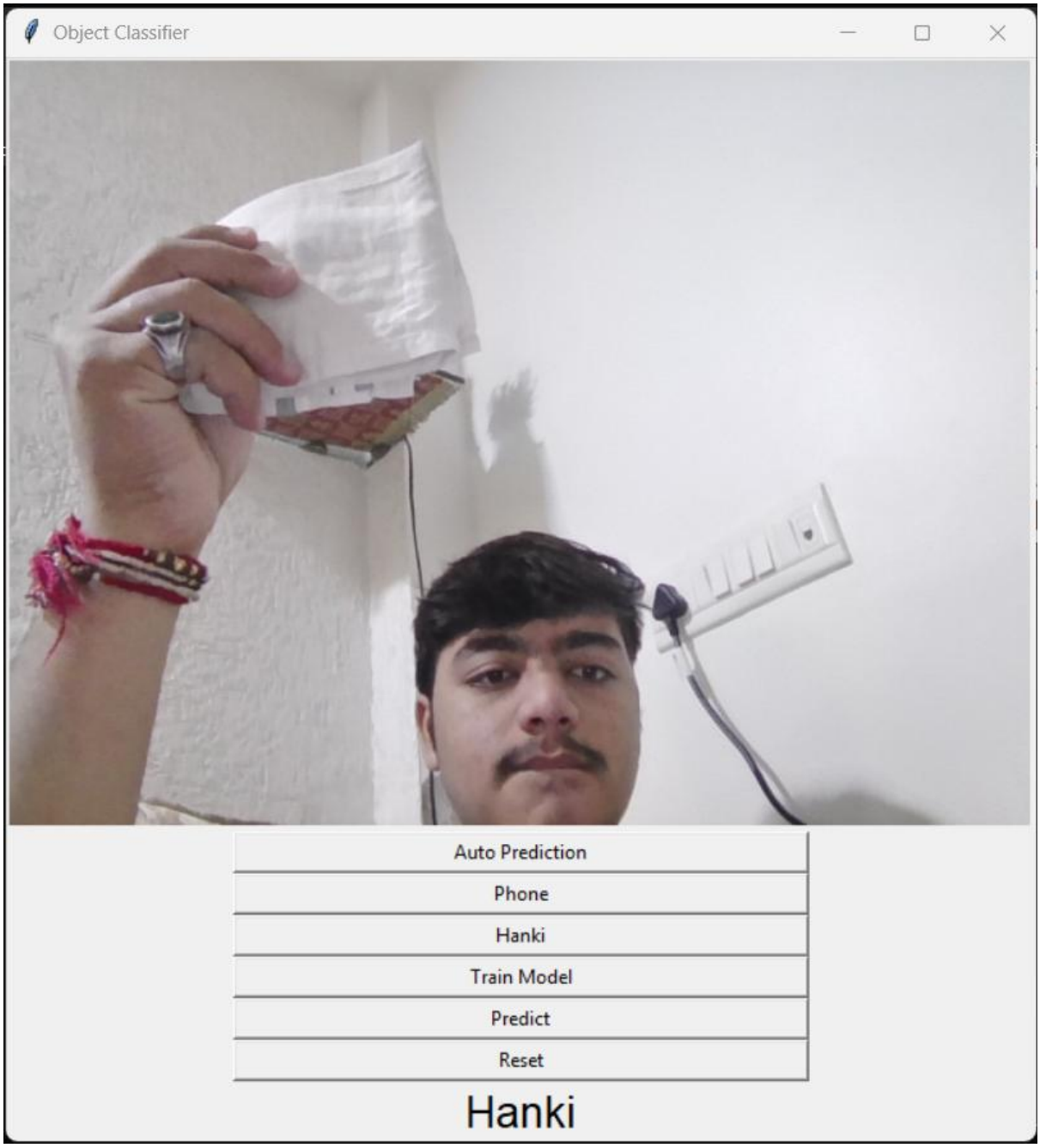
Main.py :

```
import app


def main():
    app.App(window_title="Object Classifier")


if __name__ == "__main__":
    main()
```

Output :


Object Classifier
Auto Prediction
Phone
Hanki
Train Model
Predict
Reset
Phone

Object Classifier
Auto Prediction
Phone
Hanki
Train Model
Predict
Reset
Hanki


Object Classifier
Auto Prediction
Phone
Hanki
Train Model
Predict
Reset
Hanki